

IT112: WEB SYSTEMS and TECHNOLOGIE'S

2nd Semester, AY: 2025-2026

Phase 6: Final Group Project

BentaBoard

"From bawat benta to tunay na kita—track it right."



Proposed By:

PeNiHAMiNo

BSIT 2A

Mantes, Michael B.

Obidece, Hazel Ann R.

Odo, Pearl Anne R.

Racadag, Nica Alyssa R.

Remolar, Noe Jr H.

Presented to:

Dr. Guillermo V. Red, Jr.

IT112 Professor

Introduction

The BentaBoard project is designed to help small-scale sellers, such as sari-sari store owners and market vendors, manage their business finances without the need for complex accounting tools. While previous phases focused on creating accounts, adding products, and displaying sales data, a management system is not truly complete unless the user has full control over the information they have entered. This is why Phase 6 is one of the most critical parts of our development process. In this phase, we focused on implementing the "Update" and "Delete" functionalities, which completes the full CRUD (Create, Read, Update, and Delete) cycle of our application.

Without these features, a seller would be stuck with mistakes; for example, if they accidentally typed the wrong price for a product or entered the wrong amount for a daily expense, they would have no way to fix it. By implementing the PUT method in our backend, we allow users to open a pre-filled form (using Bootstrap Modals) to edit their existing data seamlessly. Additionally, by implementing the DELETE method, we provide a way to remove outdated products or incorrect entries from the database. This phase required a tight coordination between the frontend and backend to ensure that when a user clicks "Save" or "Delete," the change is not only visible on the screen but is also permanently and correctly updated in our MongoDB database. This ensures that BentaBoard is a reliable and flexible tool for real-world business use.

Objectives

At the end of this activity, we were able to:

1. Implement PUT and DELETE endpoints in the backend using Express.
2. Create and trigger frontend JavaScript functions for update and delete actions.
3. Display modals or prefilled forms for editing.
4. Handle confirmation prompts and feedback messages for deletion.
5. Ensure frontend changes are reflected in MongoDB documents.

I. Screenshots of working edit modal or page

SELLER PART

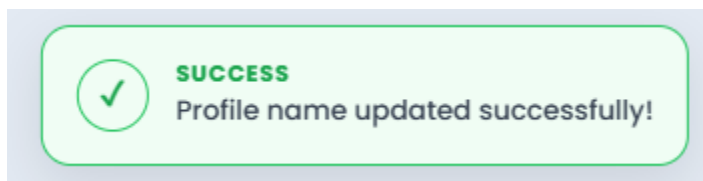
Edit Profile Modal

Profile Information
Display Name

Update Profile

Profile Information
Display Name

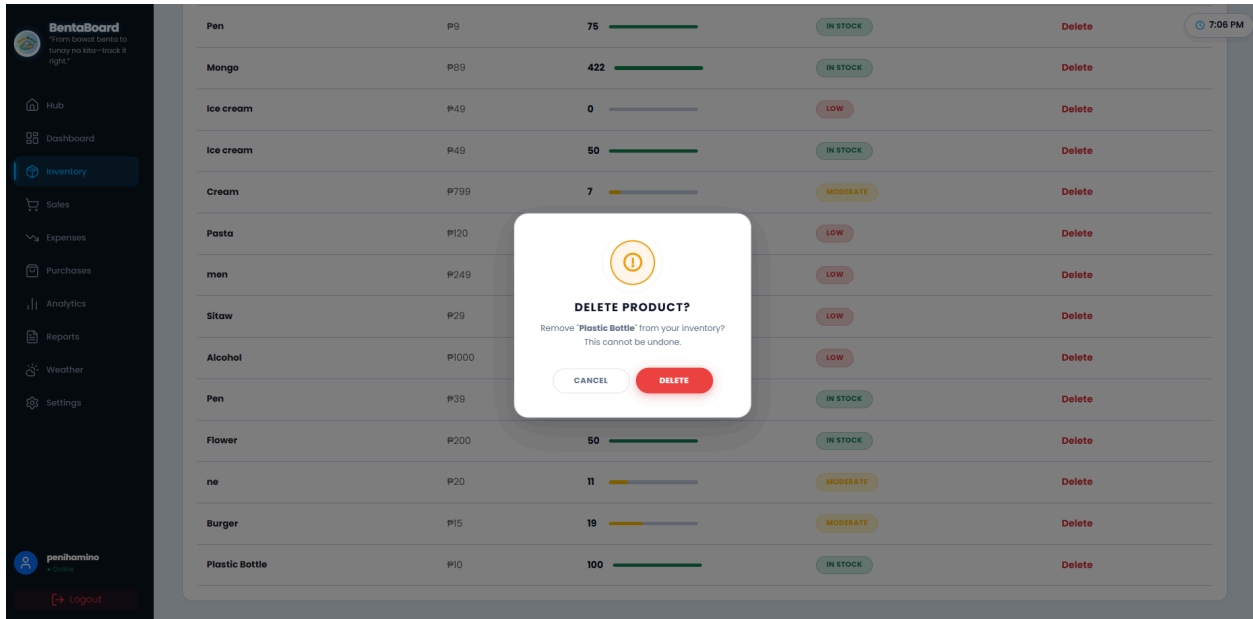
Update Profile



II. Screenshots of deletion confirmation and database updates

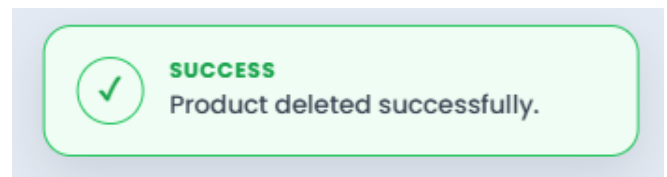
Delete Modal

- Inventory.html



Item Name	ID	Stock	Status	Action
Pen	P9	75	IN STOCK	Delete
Mongo	P89	422	IN STOCK	Delete
Ice cream	P49	0	LOW	Delete
Ice cream	P49	50	IN STOCK	Delete
Cream	P799	7	MODERATE	Delete
Pasta	P120		LOW	Delete
men	P249		LOW	Delete
Sitaw	P29		LOW	Delete
Alcohol	P1000		LOW	Delete
Pen	P39		IN STOCK	Delete
Flower	P200	50	IN STOCK	Delete
ne	P20	11	MODERATE	Delete
Burger	P15	19	MODERATE	Delete
Plastic Bottle	P10	100	IN STOCK	Delete

```
_id: ObjectId('69f5da85ce9229b433711e4f')
userId: ObjectId('69e47344c411cf6c84dd1e1c')
name: "Plastic Bottle"
price: 10
stock: 100
createdAt: 2026-05-02T11:05:41.766+00:00
__v: 0
```



Mongosh Result (Product ID Doesn't Exist)

```
> db.products.findOne({ "products.productId": ObjectId('69f5da85ce9229b433711e4f') })
< null
```

III. JavaScript snippets for edit and delete

- Delete product

```
async function deleteProduct(index) {
  if (!products[index]) return;
  bentaNotify.confirm(
    'Delete Product?',
    `Remove "<strong>${products[index].name}</strong>" from your inventory? This cannot be undone.`,
    'Delete',
    async () => {
      const productId = products[index]._id;
      if (productId) {
        try {
          await fetch(`${API_BASE_URL}/products/${productId}`, {
            method: 'DELETE',
            headers: getAuthHeaders()
          });
        } catch (err) {
          console.warn('Could not delete from DB:', err);
        }
      }
      products.splice(index, 1);
      saveData();
      renderInventory();
      populateSaleSelect();
      bentaNotify.show('success', 'SUCCESS', 'Product deleted successfully.');
```

- Update Profile

```
if (settingNameInput) {
  settingNameInput.value = currentUser;
}

if (settingsForm) {
  settingsForm.addEventListener('submit', (e) => {
    e.preventDefault();
    const newName = settingNameInput.value.trim();
    if (newName) {
      localStorage.setItem('bb_user', newName);
      currentUser = newName;
      if (userNameDisplay) userNameDisplay.innerText = newName;
      bentaNotify.show('success', 'SUCCESS', 'Profile name updated successfully!');
    }
  });
}
```

IV. Backend route and controller snippets

- Wipe User Data

```
router.delete('/wipe', protect, dataController.wipeUserData);
```

```
const wipeUserData = async (req, res) => {
  try {
    const userId = req.user.id;

    await Promise.all([
      Product.deleteMany({ userId }),
      Sale.deleteMany({ userId }),
      Expense.deleteMany({ userId }),
      Purchase.deleteMany({ userId })
    ]);

    res.status(200).json({
      message: 'All user data wiped successfully'
    });
  } catch (error) {
    console.error('Wipe error:', error);
    res.status(500).json({ message: 'Error wiping data', error: error.message });
  }
};

module.exports = { wipeUserData };
```

- Delete Products

```
router.delete('/:id', protect, deleteProduct);
```

```
const deleteProduct = async (req, res) => {
  try {
    // Only allow deleting own products (userId check prevents deleting others' data)
    const product = await Product.findOneAndDelete({
      _id: req.params.id,
      userId: req.user.id
    });

    if (!product) {
      return res.status(404).json({ message: 'Product not found or not authorized' });
    }

    res.status(200).json({ message: 'Product deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Error deleting product', error: error.message });
  }
};
```

V. GitHub commit logs

Commits on May 2, 2026		
Style: update settings view to link with modular settings.js	d52b329	<>
prianne committed 2 hours ago		
Style: update weather integration view to link with modular weatherapi.js	b6ef5be	<>
prianne committed 2 hours ago		
Style: update reporting views to link with modular analytics.js and reports.js	1807694	<>
prianne committed 2 hours ago		
Style: update sales logging view to link with modular saleslogs.js	f833b27	<>
prianne committed 2 hours ago		
Style: update financial views to link with modular expenses.js and purchases.js	f167143	<>
prianne committed 2 hours ago		
Style: update inventory view to link with modular inventory.js	b7fc6e8	<>
prianne committed 2 hours ago		
Style: update dashboard and landing pages to link with modular dashboard.js and hub.js	f29dc32	<>
prianne committed 2 hours ago		
Style: update role selection view to link with modular role-select.js	7cfced2	<>
prianne committed 2 hours ago		
Style: update authentication views to link with modular auth.js	a97a323	<>
prianne committed 2 hours ago		
Refactor: update administrative script logic to align with modular architecture	d5da797	<>
prianne committed 2 hours ago		
Refactor: migrate data visualization and analytics tracking logic to analytics.js	454cd0d	<>
prianne committed 2 hours ago		
Refactor: migrate user authentication and session management logic to auth.js	030027b	<>
prianne committed 2 hours ago		
Refactor: migrate core dashboard UI updates and state management to dashboard.js	2a8e2af	<>
prianne committed 2 hours ago		
Refactor: migrate expense tracking and financial calculation logic to expenses.js	c602004	<>
prianne committed 2 hours ago		
Refactor: migrate central navigation and hub connectivity logic to hub.js	825dd04	<>
prianne committed 2 hours ago		
Refactor: move inventory management and stock logic to inventory.js	6539c96	<>
prianne committed 2 hours ago		
Refactor: migrate application entry point and initialization to main.js	823df4e	<>
prianne committed 2 hours ago		
Refactor: extract alert and notification handling from script.js to notifications.js	885df0f	<>
prianne committed 2 hours ago		
Refactor: migrate purchase transaction and procurement logic to purchases.js	55046cb	<>
prianne committed 2 hours ago		
Refactor: migrate report generation and export logic to reports.js	59bb6f3	<>
prianne committed 2 hours ago		
Refactor: move user role selection and access control logic to role-select.js	2733848	<>
prianne committed 2 hours ago		
Refactor: move sales tracking and transaction logging to saleslogs.js	1627f73	<>
prianne committed 2 hours ago		
Refactor: migrate system configuration and user settings logic to settings.js	481c818	<>
prianne committed 2 hours ago		
Refactor: move common utility functions and shared variables to shared.js	d9fa2e2	<>
prianne committed 2 hours ago		
Refactor: migrate weather API integration from script.js to weatherAPI.js	35e21ff	<>
prianne committed 2 hours ago		
Cleanup: delete script.js after modularizing code into specific feature scripts	8de616c	<>
prianne committed 2 hours ago		

VI. Individual reflections from each group member (3–5 sentences)

Hazel Ann Obidece - Project Manager & Backend Developer

I worked on creating the PUT and DELETE routes in our Express server. My main goal was to make sure the backend could handle the requests sent from the frontend without crashing. I had to write logic that sends a success message back so the user knows their action worked. Coordinating with the frontend developer was necessary to match the API endpoints with the button clicks. Overall, this phase made me realize how much work goes into making a system truly interactive.

Noe Jr Remolar - Documentation Officer

I spent most of my time testing the system to find any bugs in the update and delete process. I tried to break the system by entering invalid data to see if the backend would catch the errors. I also made sure that the confirmation prompts worked correctly on every single product card. After finding a few small issues, I reported them to the team so we could fix them immediately. This phase taught me that testing is just as important as writing the actual code.

Michael Mantes - Database Manager

I focused on making sure the “indByIdAndUpdate” function correctly updated the MongoDB documents. It was a bit challenging to ensure that the data types, like numbers for prices, remained consistent during the update. I had to double-check that the new values were being saved in the right fields within our collections. Seeing the changes reflect immediately in the database after clicking save was very satisfying. This phase helped me understand how important data integrity is in a real application.

Nica Alyssa Racadag - FrontEnd Manager

I worked on the user interface, specifically the Bootstrap modals and the edit/delete buttons. It was tricky to make sure each button knew exactly which product ID it was supposed to edit or delete. I had to use “data-id” attributes to pass the correct information from the HTML to my JavaScript functions. I also worked on making the “Delete” confirmation pop up so users don’t delete things by accident. Seeing the buttons actually work with our real database was the best part of this week.

Pear Anne Odo - GitHub Manager

My role was to manage the repository and ensure all our new code was synced correctly. I had to handle the merging of the new update and delete branches into our main project. I made sure that no one’s work was overwritten during the process of adding the new features. Keeping the commit history clean helped us track exactly when the CRUD features were added. It was a great experience learning how to manage a team’s code in a professional way.